

# EVENTFORCE MANAGEMENT SYSTEM

*Salesforce Implementation*

## PROJECT DOCUMENTATION

College Name: Alpha College of Engineering

College Code: 2101

University Name: Anna University

**Team ID: SWTID-2026-8495**

Team Leader: Vinothan B (210123104046)

Team Member: Mohan K (210123104024)

Team Member: Subash M (210123104040)

Team Member: Sirajul Athiyab T (210123104036)

## **Table of Contents**

1. Project Overview
2. Objectives
3. Phase 1 – Requirement Analysis & Planning
4. Phase 2 – Backend Development & Configuration
5. Phase 3 – UI/UX Development & Customization
6. Phase 4 – Data Migration, Testing & Security
7. Phase 5 – Deployment, Documentation & Maintenance
8. Testing Results
9. Conclusion – Project Learnings & Outcomes

## 1. Project Overview

The EventForce Management System is a Salesforce-based Customer Relationship Management (CRM) solution designed to digitize and coordinate event management operations. The system brings client bookings, venue reservations, vendor coordination, event scheduling, feedback collection, automation, reporting, and controlled access into a single CRM environment.

The platform replaces fragmented manual activities with centralized records, automated workflows, approval processes, real-time visibility, and analytics. The implementation uses Salesforce custom objects, relationships, validation rules, Flow automation, Apex classes and triggers, a Lightning App, reports, dashboards, and layered security controls.

The data model includes six custom objects: Event, Client, Vendor, Venue, Feedback, and the Event Vendor junction object. Relationships support client and venue associations, feedback tracking, and the many-to-many relationship between events and vendors.

All features were built and tested in a Salesforce Developer Edition org, covering the complete implementation lifecycle from requirements and data modelling through automation, UI customization, data preparation, testing, security, and deployment planning.

## 2. Objectives

The primary objective is to create a comprehensive CRM platform that digitizes and automates event management operations, streamlining client bookings, venue reservations, vendor coordination, and feedback collection while providing real-time operational visibility.

### 2.1 Business Goals

- Centralize event, client, vendor, venue, and feedback information in Salesforce.
- Automate event scheduling, client reminders, cancellation approvals, and notification workflows.
- Improve data accuracy through validation rules, formula fields, lookup filters, and controlled relationships.
- Prevent operational conflicts such as double-booking a venue on the same date.
- Provide role-based access and controlled record visibility for administrators, coordinators, vendor managers, and clients.
- Provide reports and dashboards for upcoming events, budgets, vendor activity, and operational insights.
- Reduce manual processes by 60% and improve booking efficiency by 40%.

## 2.2 Expected Business Value

| Business Outcome                  | Target                                                    |
|-----------------------------------|-----------------------------------------------------------|
| Reduction in manual processes     | 60%                                                       |
| Improvement in booking efficiency | 40%                                                       |
| Client satisfaction               | Improved through timely reminders and communication       |
| Decision making                   | Data-driven insight from integrated reports and analytics |

### 3. Phase 1 – Requirement Analysis & Planning

Phase 1 establishes the business and technical requirements for the EventForce CRM. The system is planned as a single Salesforce platform for managing event planners, clients, vendors, venues, and feedback.

#### 3.1 Functional Scope

- Custom objects: Event, Client, Vendor, Venue, Feedback, and Event Vendor junction object.
- Lookup, Master-Detail, and many-to-many relationship modelling.
- Flows for reminders, feedback notifications, and approval updates.
- Validation rules and formula fields for data accuracy and automatic event budget calculation.
- Approval process for event cancellation requests with email notifications.
- Profiles, Roles, Permission Sets, and Sharing Rules for user-level access and record visibility.
- Reports and Dashboards for event summaries, vendor performance, and upcoming-event insights.

#### 3.2 Stakeholder Mapping

| Stakeholder          | Role in the System                                                    |
|----------------------|-----------------------------------------------------------------------|
| Clients              | Request event services, provide feedback, and view event updates      |
| Event Coordinators   | Manage event scheduling, client communication, and approval workflows |
| Vendors              | Handle assigned event services like catering, décor, and logistics    |
| Venue Managers       | Manage venue availability and reservations                            |
| Event Administrators | Oversee system operations, security, and data monitoring              |

#### 3.3 Execution Roadmap

- Object & Field Setup – creation of Event, Client, Vendor, Venue, Feedback, and Event Vendor objects.
- Automation Flows – record-triggered flow for event reminders.
- Validation Rules & Formula Fields – enforce data accuracy and budget calculations.
- Profiles, Roles & Permission Sets – configure user access and record visibility.
- Approval Process – automate the event cancellation workflow with notifications.
- Apex Classes & Triggers – update venue availability and prevent double bookings.
- Reports & Dashboards – analytics for upcoming events and performance insights.

## 4. Phase 2 – Backend Development & Configuration

Phase 2 builds the functional backend of EventForce. Salesforce declarative automation and Apex are combined to create a dynamic, automated, and secure CRM solution.

### 4.1 Core Data Architecture

| Object (API Name) | Key Fields                                                         | Purpose                           |
|-------------------|--------------------------------------------------------------------|-----------------------------------|
| Event__c          | Event Name, Event Date, Event Type, Event Status, Event Budget     | Event details, status and budget  |
| Client__c         | Client Name, Email, Phone, Address, Country, City                  | Client contact information        |
| Vendor__c         | Vendor Name, Email, Phone, Service Type, Status                    | Vendor details and services       |
| Venue__c          | Venue Name, Address, Location (URL), Capacity, Availability Status | Venue details and availability    |
| Feedback__c       | Feedback ID (Auto Number F-00000}, Rating, Comments                | Client ratings and comments       |
| EventVendor__c    | Master-Detail to Event and Vendor (junction)                       | Links Event ↔ Vendor many-to-many |

The screenshot shows the Salesforce Setup interface for creating a custom object named 'Event'. The page is titled 'Event' and is part of the 'Object Manager' section. The left sidebar contains a navigation menu with options: Details, Fields & Relationships, Page Layouts, Lightning Record Pages, Buttons, Links, and Actions, Compact Layouts, Field Sets, Object Limits, Record Types, Related Lookup Filters, Search Layouts, and List View Button Layout. The main content area is titled 'Custom Object Definition Edit' and includes a 'Save' button, a 'Save & New' button, and a 'Cancel' button. Below this is the 'Custom Object Information' section, which contains the following fields and options:

- Label:** Event (Example: Account)
- Plural Label:** Events (Example: Accounts)
- Starts with vowel sound:** ☒
- The Object Name is used when referencing the object via the API:** Event (Example: Account)
- Description:** (Empty text box)
- Context Sensitive Help Setting:** ☒ Open the standard Salesforce.com Help & Training window (Other option: Open a window using a Visualforce page)
- Content Name:** None (Dropdown menu)

At the bottom of the page, there is a section titled 'Enter Record Name Label and Format' and a button labeled 'Activate Windows' with the text 'Go to Settings to activate Windows.'

Figure 1 – Event custom object creation

The singular and plural labels are used in tabs, page layouts, and reports. Be careful when changing the name or label as it may affect existing integrations and merge templates.

Label:  Example: Account

Plural Label:  Example: Accounts

Starts with vowel sound: ☐

The Object Name is used when referencing the object via the API.

Object Name:  Example: Account

Description:

Context Sensitive Help Setting: ☒ Open the standard Salesforce.com Help & Training window ☐ Open a window using a Visualforce page

Content Name:

**Enter Record Name Label and Format**

The Record Name appears in page layouts, key lists, related lists, lookups, and search results. For example, the Record Name for Account is "Account Name" and for Case it is "Case Number". Note that the Record Name field is always called "Name" when referenced via the API.

Record Name:  Example: Account Name

Data Type:  Warning: If you plan to insert a high volume of records in this object, via the API for example, use the Text data type.

Display Format:  Example: A-0000 What Is This?

Activate Windows  
Go to Settings to activate Windows.

Figure 2 – Feedback custom object with Auto Number record name

## 4.2 Fields Created per Object

| Object   | Field                         | Data Type                 | Values / Notes                                                                                                            |
|----------|-------------------------------|---------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Event    | Event Type                    | Picklist                  | Wedding, Corporate, Birthday, Anniversary, Festival, Concert, Other                                                       |
| Event    | Event Date                    | Date                      | Date the event takes place                                                                                                |
| Event    | Event Status                  | Picklist                  | Planned, Confirmed, Completed, Pending Cancellation, Canceled, Rejected                                                   |
| Event    | Event Budget                  | Formula (Currency)        | Derived from Event Type                                                                                                   |
| Client   | Email / Phone / Address       | Email / Phone / Text Area | Contact details                                                                                                           |
| Client   | Country / City                | Picklist                  | 14 countries; 24 cities                                                                                                   |
| Vendor   | Service Type                  | Multi-select Picklist     | Catering, Décor, Photography, Videography, Lighting, Stage Setup, Makeup Artist, DJ/Music, Transportation, Hosting/Anchor |
| Vendor   | Status                        | Picklist                  | Available, Booked, Cancelled                                                                                              |
| Venue    | Address / Location / Capacity | Text Area / URL / Number  | Venue details and map link                                                                                                |
| Venue    | Availability Status           | Picklist                  | Available, Reserved, Not Available                                                                                        |
| Feedback | Rating                        | Picklist                  | 1, 2, 3, 4, 5                                                                                                             |

|          |          |                |                 |
|----------|----------|----------------|-----------------|
| Feedback | Comments | Long Text Area | Client comments |
|----------|----------|----------------|-----------------|

Edit Event Custom Field  
Event Budget

Custom Field Definition Edit Save Quick Save Cancel

Field Information

Field Label

Field Name

Description

Help Text

Data Owner

Field Usage

Data Sensitivity Level

Compliance Categorization

| Available | Chosen |
|-----------|--------|
| PII       |        |
| HIPAA     |        |
| GDPR      |        |
| PCI       |        |

Figure 3 – Event Budget formula field (syntax check passed)

### 4.3 Relationships

| Relationship         | Type          |
|----------------------|---------------|
| Event → Client       | Lookup        |
| Event → Venue        | Lookup        |
| Feedback → Event     | Lookup        |
| Feedback → Client    | Lookup        |
| EventVendor → Event  | Master-Detail |
| EventVendor → Vendor | Master-Detail |

Salesforce does not support direct many-to-many relationships, so the EventVendor\_\_c junction object with two Master-Detail fields links Events and Vendors.

### 4.4 Lookup Filter (Feedback → Event)

A lookup filter on the Feedback object's Event field ensures that after a client is selected, only that client's events appear in the Event lookup. Filter criteria: Event: Client\_\_c equals Field: Feedback: Client\_\_c.

**Lookup Options**

Related To:  Child Relationship Name:

Related List Label:

Required: ☐ Always require a value in this field in order to save a record.

What to do if the lookup record is deleted? ☒ Clear the value of this field. You can't choose this option if you make this field required.

☐ Delete this record also. This allows users to delete large numbers of records without regard for sharing or visibility constraints.

☐ Don't allow deletion of the lookup record that's part of a lookup relationship.

**Lookup Filter**

Optionally, create a filter to limit the records available to users in the lookup field. [Tell me more!](#)

[Hide Filter Settings](#)

Filter Criteria:  Operator:  Value / Field:

AND

[Add Filter Logic...](#)

Filter Type: ☒ **Required.** The user-entered value must match filter criteria.

☐ **Optional.** The user can remove the filter or enter values that don't match criteria.

Lookup Window Text:

Active: ☒ Enable this filter.

Figure 4 – Lookup filter on the Feedback → Event field

## 4.5 Validation Rule – Email Format

A validation rule on the Client object blocks saves when the email format is invalid.

$NOT(REGEX(Email\_c, "[a-zA-Z0-9.]+\@[a-zA-Z0-9.]+\.[a-zA-Z]{2,}\$"))$

Error message: "Please Enter Valid Email Address", shown on the Email field.

**Client Validation Rule**

Define a validation rule by specifying an error condition and a corresponding error message. The error condition is written as a Boolean formula expression that returns true or false. When the formula expression returns true, the save will be aborted and the error message will be displayed. The user can correct the error and try again.

**Validation Rule Edit**

Rule Name:

Active: ☒

Description:

**Error Condition Formula**

Example: `Discount_Percent > 30`

Display an error if Discount is more than 30%

If this formula expression is true, display the text defined in the Error Message area

[Insert Field](#) [Use Formula Assistant](#) [Insert Operator](#)

`NOT (REGEX (Email__c, "[a-zA-Z0-9.]+\@[a-zA-Z0-9.]+\.[a-zA-Z]{2,}\$"))`

**Functions**

ABS: Returns the absolute value of a number, a number without its sign.

ACOS: Returns the arccosine of the argument.

ADDMONTHS: Returns the date that is the specified number of months after the start date.

AND: Returns true if all arguments are true.

ASCII: Returns the ASCII value of the argument.

ASIN: Returns the arcsine of the argument.

Figure 5 – Email\_Valid\_Address validation rule

## 4.6 Approval Process – Event Cancellation

The Event\_Cancellation\_Process ensures an event is canceled only after manager approval, with automatic notifications to all parties. Four classic email templates were created first:

| Template                                | Subject                                         | Used For                    |
|-----------------------------------------|-------------------------------------------------|-----------------------------|
| Event Cancellation Request Notification | Approval Request: Cancel Event {!Event__c.Name} | Alert on submission         |
| Email to Client After Cancellation      | Event Cancellation Notice – {!Event__c.Name}    | Client notified of approval |

|                                              |                                                  |                             |
|----------------------------------------------|--------------------------------------------------|-----------------------------|
| <b>Approval Notification to Event Owner</b>  | Event Cancellation Approved:<br>{!Event__c.Name} | Owner notified of approval  |
| <b>Rejection Notification to Event Owner</b> | Event Cancellation Rejected:<br>{!Event__c.Name} | Owner notified of rejection |

- Entry criteria: Event Status equals Pending Cancellation.
- Approver: assigned automatically to the submitter's manager.
- Initial submitter: Event Owner.
- Final approval actions: field update Event Status = Canceled; email alert to Event Owner.
- Final rejection actions: field update Event Status = Rejected; email alert to Event Owner.
- Approval step: Manager Review, activated after configuration.

The screenshot shows the 'Classic Email Templates' setup interface. The template is titled 'Event Cancellation Request Notification'. It includes fields for 'Subject' (set to 'Event Cancellation Request'), 'To' (set to 'Event Owner'), and 'From' (set to 'Savitha KM'). The 'Description' field contains the text: 'An approval request from team submitted to cancel the event. - Event Name: {!Event\_\_c.Name} - Event Date: {!Event\_\_c.EventDate} - Thank you! (Client: {!Event\_\_c.ClientName})'. The 'Email Body' field contains a placeholder for the email content. The 'Template Name' is 'Event Cancellation Request' and the 'Category' is 'Event Cancellation Request'.

Figure 6 – Classic email template: cancellation request notification

The screenshot shows the 'Event\_Cancellation\_Process' configuration page. It includes a 'Process Definition Detail' section with the following information:

| Field                              | Value                                           |
|------------------------------------|-------------------------------------------------|
| Process Name                       | Event_Cancellation_Process                      |
| Unique Name                        | Event_Cancellation_Process                      |
| Description                        | Event: Event Status EQUALS Pending Cancellation |
| Entry Criteria                     | Administrator OR Current Approver               |
| Record Editability                 | Allow Submitters to Recall Approval Requests    |
| Approval Assignment Email Template | Event_Cancellation_Request_Notification         |
| Initial Submitters                 | Event Owner                                     |
| Created By                         | savitha.km, 9/29/2026, 2:13 AM                  |
| Modified By                        | savitha.km, 10/1/2026, 1:49 AM                  |

Figure 7 – Event\_Cancellation\_Process activated

## 4.7 Flow – 3-Day Reminder Email

A record-triggered flow sends the client a reminder email three days before a confirmed event. Configuration:

- Object: Event; trigger: record is created or updated; condition: Event Status = Confirmed.
- Optimize for: Actions and Related Records.

- Scheduled path 3\_Days\_Before: time source Event\_\_c: Event Date, offset 3 days before.
- Text templates Subject and Body composed with record merge fields.
- Send Email action (Alert\_Client\_3Day\_Reminder): recipient = Client Email, CC = Event Owner.
- Flow label: Client Reminder – 3 Days Before; saved and activated.

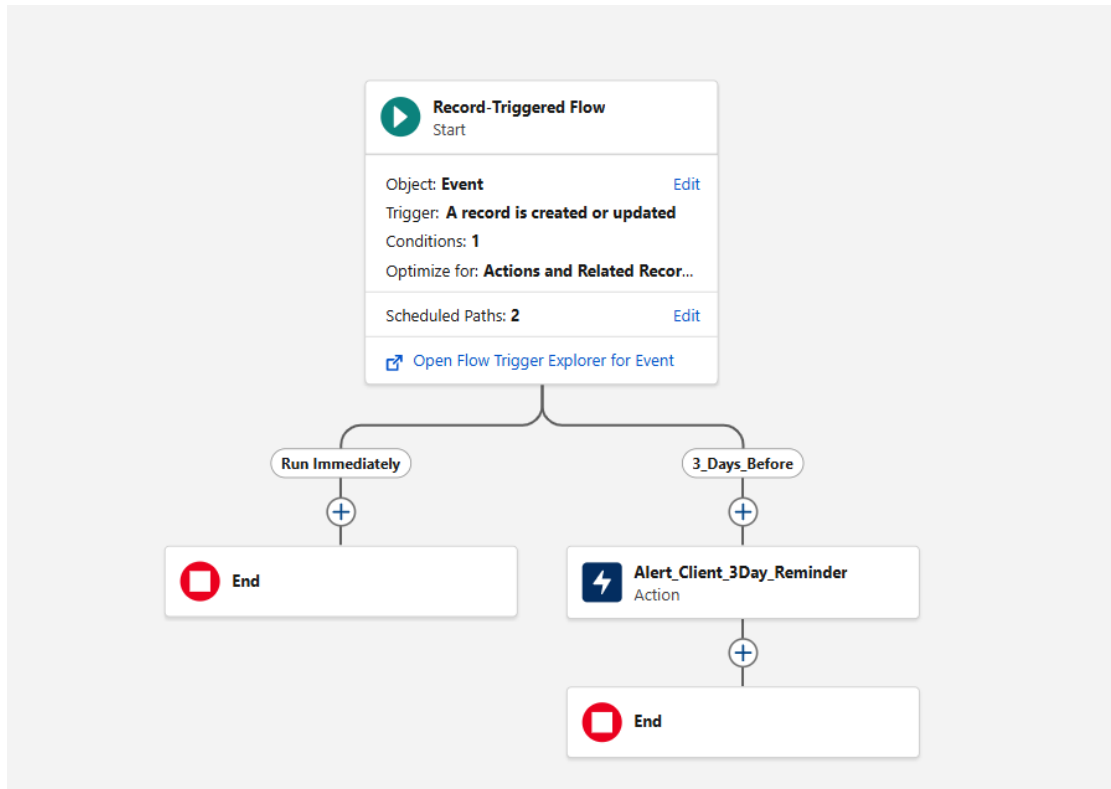


Figure 8 – Client Reminder – 3 Days Before flow

#### 4.8 Apex – Venue Status Automation

Scenario: when an Event with a Venue is confirmed, the venue's Availability Status becomes Reserved; when the event is canceled it returns to Available. Logic is held in a helper class called from a thin trigger.

```

public with sharing class VenueStatusHelper { ... }

trigger EventTrigger13 on Event__c (after insert, after update) {
    VenueStatusHelper.updateVenueStatus(Trigger.new); }
  
```

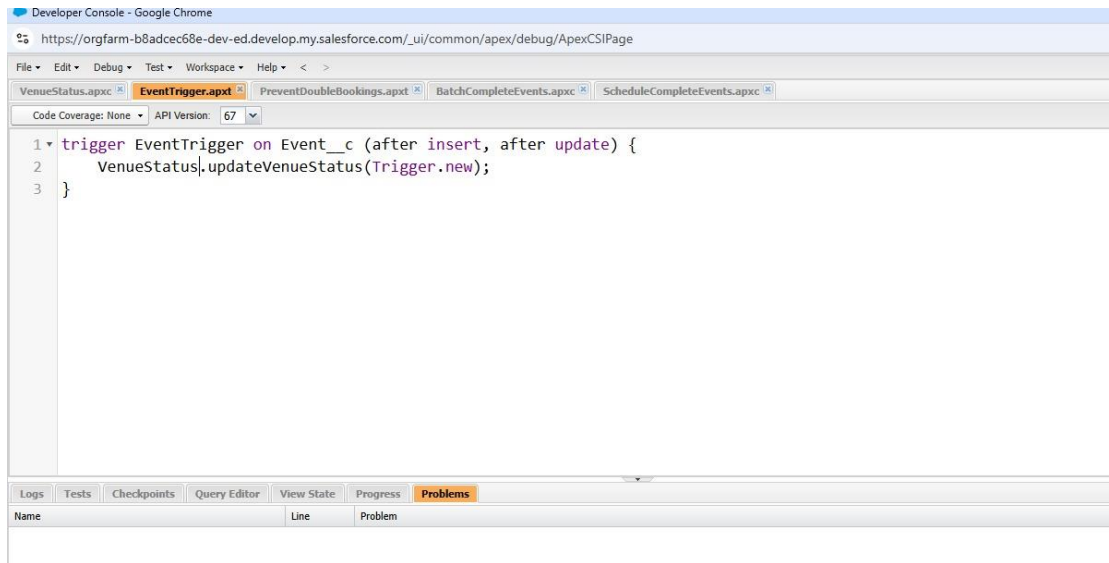


Figure 9 – EventTrigger13 calling VenueStatusHelper

## 4.9 Apex – Prevent Double Booking

A before-insert / before-update trigger gathers the venues involved, fetches existing events at those venues in a single bulk-safe query, and raises an error when another event already holds the same date: "This Venue is already booked on this date."

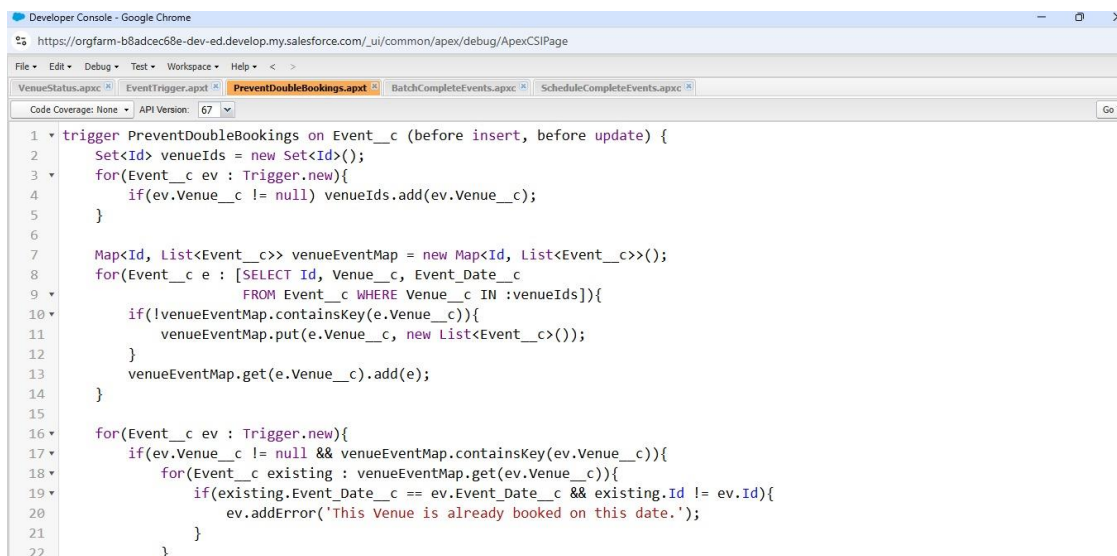


Figure 10 – PreventDoubleBooking trigger

## 4.10 Batch & Scheduled Apex – Nightly Completion

BatchCompleteEvents queries events where Event Date < TODAY and status ≠ Completed and marks them Completed. The ScheduleCompleteEvents schedulable class launches the batch nightly via Setup → Apex Classes → Schedule Apex (job: Daily Event Completion, 8:00 PM, every day).

```
Developer Console - Google Chrome
orgfarm-b8adcec68e-dev-ed.develop.my.salesforce.com/_ui/common/apex/debug/ApexCSIPage

File Edit Debug Test Workspace Help < >
VennasStatus.apex EventTrigger.apex PreventDoubleBookings.apex BatchCompleteEvents.apex ScheduleCompleteEvents.apex
Code Coverage: None API Version: 67

1 global class BatchCompleteEvents implements Database.Batchable<Object> {
2     global Database.QueryLocator start(Database.BatchableContext bc){
3         return Database.getQueryLocator(
4             'SELECT Id, Event_Date__c, Event_Status__c FROM Event__c WHERE Event_Date__c < TODAY AND Event_Status__c != \'Completed\''
5         );
6     }
7     global void execute(Database.BatchableContext bc, List<Event__c> scope){
8         for(Event__c ev : scope){
9             ev.Event_Status__c = 'Completed';
10        }
11        update scope;
12    }
13    global void finish(Database.BatchableContext bc){
14        System.debug('Past events marked as Completed');
15    }
16 }
```

Figure 11 – BatchCompleteEvents class

The screenshot shows the Salesforce Setup page for Apex Classes. The left sidebar contains a navigation menu with options like Email, Custom Code, Apex Classes, Apex Settings, Apex Triggers, Application Test Execution, Application Test History, Environments, and Jobs. The main content area is titled 'Apex Classes' and shows a configuration for a scheduled job named 'Daily Event Completion'. The job is scheduled using the 'Schedule Builder' and is set to run weekly on all days of the week (Sunday through Saturday). The start date is 10/1/2020 and the end date is 10/1/2020. The preferred start time is set to 'None'.

Job Name: Daily Event Completion  
Apex Class: BatchCompleteEvents

Schedule Using: Schedule Builder (selected) | Cron Expression

Frequency: Weekly (selected) | Monthly

Recurs every week on:  
☒ Sunday  
☒ Monday  
☒ Tuesday  
☒ Wednesday  
☒ Thursday  
☒ Friday  
☒ Saturday

Start: 10/1/2020 | 10/1/2020  
End: 11/1/2020 | 10/1/2020  
Preferred Start Time: --None--

Exact start time will depend on job queue activity.

Figure 12 – Scheduled job: Daily Event Completion

The EventForce Operations Dashboard visualizes the report as a donut chart (max 6 values, light theme) showing the upcoming events overview.

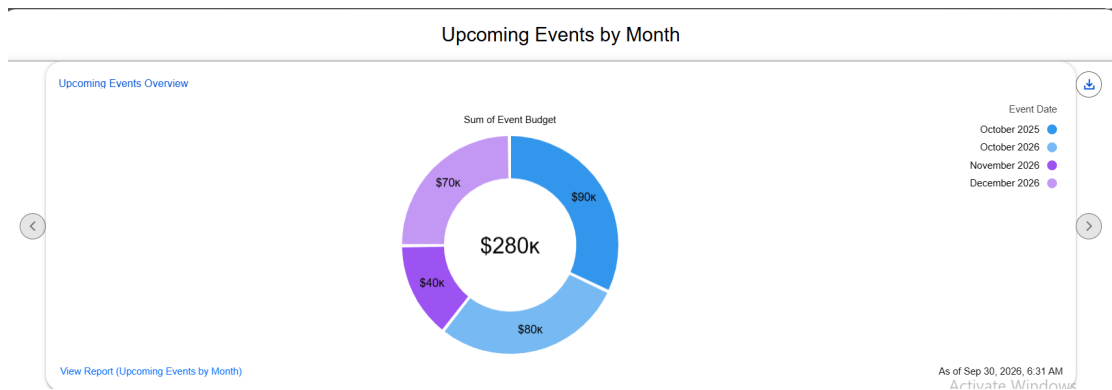


Figure 15 – EventForce Operations Dashboard

## 6. Phase 4 – Data Migration, Testing & Security

### 6.1 Data Migration

Records for Events, Clients, Vendors, Venues, and Feedback were imported using the Salesforce Data Import Wizard after pre-validation checks to avoid duplicates and maintain data integrity.

### 6.2 Profiles

| Object   | Event Coordinator | Vendor Manager | Client Profile         | Event Admin |
|----------|-------------------|----------------|------------------------|-------------|
| Event    | CRUD              | Read Only      | Read Only (own events) | Full        |
| Client   | Read, Edit        | Read Only      | No access              | Full        |
| Vendor   | Read              | CRUD           | No access              | Full        |
| Venue    | Read              | Read Only      | No access              | Full        |
| Feedback | Create, Read      | Read Only      | Create, Read           | Full        |

Event Admin was cloned from System Administrator (inherits Modify All Data and full CRUD). The other three profiles were cloned from Standard Platform User: Event Planner set as default app, session timeout 2 hours, passwords never expire, minimum password length 8.

The screenshot shows the Salesforce Setup Profiles page for the 'Event Coordinator' profile. The 'Custom Object Permissions' section is expanded, showing permissions for Clients, Events, Event Vendors, and Feedbacks. The 'Session Settings' section shows 'Session Times Out After' set to '2 hours of inactivity' and 'Session Security Level Required at Login' set to '--None--'. The 'Password Policies' section shows 'User passwords expire in' set to 'Never expires', 'Enforce password history' set to '3 passwords remembered', 'Minimum password length' set to '8', and 'Password complexity requirement' set to 'Must include alpha and numeric characters'.

|               | Basic Access                        |                                     |                                     |                                     | Data Administration      |                          |                          |
|---------------|-------------------------------------|-------------------------------------|-------------------------------------|-------------------------------------|--------------------------|--------------------------|--------------------------|
|               | Read                                | Create                              | Edit                                | Delete                              | View All Records         | Modify All Records       | View All Fields          |
| Clients       | <input checked="" type="checkbox"/> | <input type="checkbox"/>            | <input checked="" type="checkbox"/> | <input type="checkbox"/>            | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> |
| Events        | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> |
| Event Vendors | <input type="checkbox"/>            | <input type="checkbox"/>            | <input type="checkbox"/>            | <input type="checkbox"/>            | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> |
| Feedbacks     | <input checked="" type="checkbox"/> | <input checked="" type="checkbox"/> | <input type="checkbox"/>            | <input type="checkbox"/>            | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> |

**Session Settings**

Session Times Out After: 2 hours of inactivity

Session Security Level Required at Login: --None--

**Password Policies**

User passwords expire in: Never expires

Enforce password history: 3 passwords remembered

Minimum password length: 8

Password complexity requirement: Must include alpha and numeric characters

Figure 16 – Event Coordinator profile object permissions

### 6.3 Roles

| Role              | Reports To  |
|-------------------|-------------|
| Event Admin       | CEO         |
| Event Coordinator | Event Admin |

|                       |             |
|-----------------------|-------------|
| <b>Vendor Manager</b> | Event Admin |
| <b>Client</b>         | Event Admin |

## 6.4 Users

Users were created with matching role, license, and profile: Event Admin (Salesforce license) and Event Coordinator, Vendor Manager, and Client (Salesforce Platform license). Note: a Developer Edition org has limited Salesforce licenses — deactivate an unused user or change the license type if none is free.

The screenshot shows the 'User Edit' page in Salesforce. The 'General Information' tab is active. The user's details are as follows:

- First Name:** savitha
- Last Name:** km
- Alias:** sav
- Email:** savithachitra1809@gmail.com
- Username:** savithachitra1809.6296a7e
- Nickname:** User179058589916372686
- Title:**
- Company:** alpha college of engineering
- Department:**
- Division:**

On the right side, the 'Role' is set to '<None Specified>'. The 'User License' is 'Salesforce' and the 'Profile' is 'System Administrator'. The 'Active' checkbox is checked. Other roles like 'Marketing User', 'Offline User', 'Knowledge User', 'Flow User', 'Service Cloud User', 'Site.com Contributor User', 'Site.com Publisher User', and 'WDC User' are unchecked. The 'Data.com User Type' is set to '--None--' and the 'Data.com Monthly Addition Limit' is 'Default Limit (300)'. An 'Activate Windows' watermark is visible in the bottom right corner.

Figure 17 – All Users list

## 6.5 Permission Set – Feedback Manager

A Feedback Manager permission set grants Read/Create/Edit/Delete on the Feedback object and is assigned to the Event Coordinator user, extending permissions without changing the profile.

Permission Set

# Feedback Manager

Permission Set Overview > Object Settings
**Feedbacks**

## Feedbacks

### Tab Settings

| Available                | Visible                                                   |
|--------------------------|-----------------------------------------------------------|
| <input type="checkbox"/> | <input type="checkbox"/> <input type="button" value="i"/> |

### Object Permissions

| Permission Name    | Enabled                             |
|--------------------|-------------------------------------|
| Read               | <input checked="" type="checkbox"/> |
| Create             | <input checked="" type="checkbox"/> |
| Edit               | <input checked="" type="checkbox"/> |
| Delete             | <input checked="" type="checkbox"/> |
| View All Records   | <input type="checkbox"/>            |
| Modify All Records | <input type="checkbox"/>            |
| View All Fields    | <input type="checkbox"/>            |

Figure 18 – Feedback Manager permission set

## 6.6 Sharing Settings

Organization-Wide Defaults for Event were set to Private (internal and external). An owner-based sharing rule Event\_Sharing\_For\_Vendors shares records owned by the Event Coordinator role with the Vendor Manager role at Read Only access — sharing rules extend access and never restrict it.

 **Sharing Settings**

Setup

### Environment Sharing Rule

Use sharing rules to make automatic exceptions to your organization-wide sharing settings for defined sets of users.

Note: "Roles and subordinates" includes all users in a role, and the roles below that role.

You can use sharing rules only to grant wider access to data, not to restrict access.

Step 1: Rule Name

Label

Event\_Sharing\_For\_Vendor

Rule Name

Event\_Sharing\_For\_Vendor

Description

Step 2: Select your rule type

Rule Type

☒ Based on record owner ☐ Based on criteria

Step 3: Select which records to be shared

Environment: owned by members of

Public Groups

-- Select One --

Step 4: Select the users to share with

Share with

Public Groups

-- Select One --

Step 5: Select the level of access for the users

Access Level

Read Only

Figure 19 – Event\_Sharing\_For\_Vendors sharing rule

## **7. Phase 5 – Deployment, Documentation & Maintenance**

### **7.1 Deployment Strategy**

All configurations, customizations, and automations were built and tested within a Salesforce Developer Edition org. Production deployment was not performed because Developer Edition orgs are standalone environments; real deployments require sandbox environments, change sets, or Salesforce DevOps tooling (GitHub CI/CD, Salesforce DX).

A real enterprise deployment would follow this path:

- Build and unit-test in a Developer or Developer Pro sandbox.
- Promote metadata (objects, fields, flows, Apex, approvals, reports, dashboards) to a Partial or Full sandbox for UAT via change sets or Salesforce DX.
- Run Apex tests — a minimum of 75% code coverage is required to deploy to production.
- Deploy in a low-usage window, then activate flows, approvals and scheduled jobs and assign permission sets.
- Load production data, train users, and provide post-go-live support.

### **7.2 Maintenance, Monitoring & Troubleshooting**

- Keep event, client, vendor, venue, and feedback records accurate and current.
- Monitor Flows, Apex jobs, scheduled jobs, and pending approvals for failures.
- Resolve user-access errors by checking profiles, permission sets, roles, and sharing rules.
- Debug Apex triggers and Flow logic using debug logs and the Developer Console.
- Adjust validation rules and sharing settings as business requirements change; run periodic duplicate checks.

### **7.3 Documentation Summary**

This document records the purpose, data model, configuration, automation, security, and testing of EventForce. It covers the business overview and objectives, phase-wise implementation, the data model and relationships, automation components (Flows, Approval Process, validation rules, Apex), the security model (profiles, roles, permission sets, OWD, sharing rules), and testing results with screenshots — providing a blueprint for future developers and administrators and supporting training, maintenance, and knowledge transfer.

## **8. Testing Results**

Functional and performance testing was carried out in the Developer Edition org across configuration, automation, and security. All features behaved as expected.

| #  | Area               | Test Scenario                                        | Expected Result                                          |
|----|--------------------|------------------------------------------------------|----------------------------------------------------------|
| 1  | Validation rule    | Save a Client with an invalid email such as 'abc@'   | Save blocked with 'Please Enter Valid Email Address'     |
| 2  | Formula field      | Create an event of each Event Type                   | Event Budget fills in per the CASE formula               |
| 3  | Double booking     | Create two events at the same venue on the same date | Error: 'This Venue is already booked on this date.'      |
| 4  | Venue status       | Set an event to Confirmed, then Canceled             | Venue becomes Reserved, then Available                   |
| 5  | Approval process   | Set status to Pending Cancellation and submit        | Approval request and alert emails sent                   |
| 6  | Approval – approve | Approver approves the request                        | Status = Canceled; owner notified                        |
| 7  | Approval – reject  | Approver rejects the request                         | Status = Rejected; owner notified                        |
| 8  | Flow               | Confirm an event 3+ days ahead with a client email   | Reminder sent three days before the event date           |
| 9  | Batch Apex         | Run the batch with past-dated events                 | Past events marked Completed                             |
| 10 | Lookup filter      | Pick a client on Feedback, open Event lookup         | Only that client's events listed                         |
| 11 | Security           | Log in as each user type                             | Each user sees only permitted tabs, objects, and records |
| 12 | Sharing rule       | Vendor Manager views coordinator-owned events        | Events visible as Read Only                              |

## 9. Conclusion – Project Learnings & Outcomes

The EventForce Management System demonstrates how Salesforce can be configured as an integrated CRM platform for event-management operations. The project covers the complete implementation lifecycle from requirements and data modelling through automation, UI customization, data preparation, testing, security, and deployment planning.

Key learnings include:

- Designing a scalable data model, including a junction object for many-to-many links.
- Combining declarative tools (Flows, approvals, validation rules) with Apex for advanced business logic.

- Writing bulk-safe Apex triggers and using Batch and Schedulable Apex for asynchronous processing.
- Layering security with profiles, roles, permission sets, OWD, and sharing rules.
- Turning operational data into decisions through reports and dashboards.
- Mapping an academic Developer Org build to real deployment, maintenance, and documentation practice.

The completed design provides a structured foundation for managing events, clients, vendors, venues, and feedback in one CRM environment, and the documented deployment and maintenance practices provide a path toward a controlled production rollout.

**Thank You**